

Ch10 数组和指针

对应《C Primer Plus（第6版）》第 10 章 · 本章目标：30 分钟读懂 + 60 分钟练完 · 不覆盖：指针的高级用法（函数指针、指针数组、多级指针）

10.1 数组：同一类数据的集合

数组就是一排连续排列的同类型变量。声明 `int a[5]`；相当于一次申请了 5 个 `int` 的格子，用下标访问：`a[0]` 到 `a[4]`（下标从 0 开始，不是 1！）。

```
/* ch10_01_array.c - 数组的声明与初始化 */
#include <stdio.h>

int main(void)
{
    int a[5] = {10, 20, 30, 40, 50};    /* 声明 + 初始化 */
    int i;

    for (i = 0; i < 5; i++)
        printf("a[%d] = %d\n", i, a[i]);
    return 0;
}
```

编译：gcc -Wall -Wextra -o a.exe a.c

数组越界的可怕之处：`a[5]` 不报编译错——C 语言不检查下标。它只是悄悄读/写了数组后面那块内存，轻则读到垃圾值，重则改坏别的变量、程序崩溃。这是 C 最著名的坑之一，写循环边界时多看一眼。

10.2 数组名就是地址（Ch4 伏笔兑现）

Ch4 说过「数组名是起始位置的地址」。现在正式说：**数组名 `a` 的值是数组第一个元素 `a[0]` 的地址。**`&a[0]` 和 `a` 是同一个地址。

```
/* ch10_02_addr.c - 数组名与地址 */
#include <stdio.h>

int main(void)
{
    int a[3] = {7, 8, 9};

    printf("a          = %p\n", (void*)a);
}
```

```

    printf("&a[0]    = %p\n", (void*)&a[0]);
    printf("&a[1]    = %p\n", (void*)&a[1]);
    return 0;
}

```

%p 打印地址：相邻元素地址相差 sizeof(int) = 4 字节。

10.3 指针：专门存地址的变量

指针是一个变量，里面存的是**另一个变量的地址**。声明 `int *p`；读作「p 是一个指向 int 的指针」。配合两个运算符：

- **&**（取地址）：`p = &n`；把 n 的地址存进 p。
- *****（解引用）：`*p` 表示「p 指向的那个变量」。`*p = 5`；等价于给 n 赋 5。

```

/* ch10_03_pointer.c - 指针基础 */
#include <stdio.h>

int main(void)
{
    int n = 42;
    int *p = &n;      /* p 存 n 的地址 */

    printf("n = %d\n", n);
    printf("*p = %d\n", *p); /* 通过指针读 n */
    *p = 100;           /* 通过指针改 n */
    printf("改后 n = %d\n", n);
    return 0;
}

```

新手坑 1：指针必须先指向某处再解引用。`int *p; *p = 5;` —— p 没初始化，指向哪里未知，写进去就是**野指针崩溃**。`-Wall -Wextra` 会提示 `used uninitialized`。

10.4 数组和指针的等价关系：a[i] 就是 *(a+i)

这是本章**最核心**的一个公式：

```
a[i] == *(a + i)
```

`a+i` 是「从数组开头往后数 i 个元素的那个地址」，`*(a+i)` 就是那个地址上的值。所以下标只是指针运算的语法糖。理解它，就理解了 C 数组的本质。

```

/* ch10_04_equiv.c - 数组与指针等价 */
#include <stdio.h>

```

```

int main(void)
{
    int a[4] = {2, 4, 6, 8};
    int *p = a;          /* p 也指向数组开头 */

    printf("a[2] = %d, *(a+2) = %d\n", a[2], *(a + 2));
    printf("p[2] = %d, *(p+2) = %d\n", p[2], *(p + 2));
    return 0;
}

```

10.5 函数传数组：传的是地址，不是副本

函数形参写成数组，实际拿到的是**指向数组开头的指针**——数组不会被复制。所以函数里改数组，外面的数组会跟着变。

```

/* ch10_05_func.c - 函数与数组 */
#include <stdio.h>

int sum(const int arr[], int n) /* const 承诺不修改数组内容 */
{
    int i, total = 0;
    for (i = 0; i < n; i++)
        total += arr[i];
    return total;
}

int main(void)
{
    int a[5] = {1, 2, 3, 4, 5};
    printf("总和 = %d\n", sum(a, 5));
    return 0;
}

```

为什么 const 是好习惯：函数不打算改数组时就写 `const int arr[]`，编译器会拦住你误改——这是「给自己设的护栏」，萝在 Ch9 函数也讲过这个思路。

10.6 常见坑集中营

1. **下标越界：**`a[n]` 不报错、悄悄越界（见 10.1）。
2. **野指针解引用：**指针没初始化就用 `*p`（见 10.3）。
3. **& 和 * 混淆：**`p = &n` 存地址；`*p = 5` 取值。反了就是类型不匹配警告。

4. **以为传数组是复制**：函数改形参数组会真的改掉外面——需要副本时自己手动拷（第 11 章 strcpy 思路同理）。

10.7 一分钟自测（答案见练习册）

判断对错：

- ① `int a[5] = {1, 2, 3, 4, 5}; printf("%d", a[5]);` 是安全的，会输出 5。
- ② `int *p; *p = 10;` 是合法写法。
- ③ 函数形参 `int arr[]` 实际是 `int *arr`。

10.8 练习指引

做完本章内容后，去练习册完成 Ch10 配套练习；同时打开《C Primer Plus（第6版）》第 10 章，完成**复习题 2、4、6**（题号对应主教材）。本章是考试和后续课程（数据结构）的地基，改错题请务必先自己找 bug。

本讲义为自写原创讲解，仅作课堂伴读；示例代码均可用 `gcc -Wall -Wextra` 编译。原书题目请以正版教材为准。